

Keywords and options for the generalized internal coordinates in *Gaussian*: GDV User's reference guide

Last updated: December 17, 2022

Software version: ES64L-GDVRvJ.22+

Abstract. The generalized internal coordinates hereinafter referred to as GICs [Marenich, A. V.; Hratchian, H. P.; Sonnenberg, J. L.; Frisch, M. J. Implementation of the generalized internal coordinates for molecular geometry, energy gradients, and force constants. 249th National ACS Meeting, Denver, CO, March 20-27, 2015] can be specified in the user's input file with **Opt** keyword for geometry optimizations and with **Geom** keyword for any other case that involves internal coordinates. Note that the GICs can be the same regular bonds, angles, and dihedral angles generated by the older algorithm (*Peng96*) [Peng, C.; Ayala, P. Y.; Schlegel, H. B.; Frisch, M. J. Using redundant internal coordinates to optimize equilibrium geometries and transition states. *J. Comp. Chem.*, 1996, 17, 49-56]. However, these new coordinates are internally stored and manipulated as generalized ones. For example, relevant analytical derivatives with respect to Cartesian coordinate displacements can be calculated automatically via an auto differentiation engine. The GICs are more flexible and, in principle, can be any combination of standard mathematical functions. This manual supplements the current description of **Geom** and **Opt** in the *Gaussian 16 (G16)* User's reference with the GIC-related options described below. See the *G16* reference for the remaining options associated with the **Geom** and **Opt** keywords that are not discussed here.

COORDINATE SYSTEM SELECTION OPTIONS

This section describes regular GIC-related options. There are some additional (experimental) features that can be invoked using certain IOps described below. A list of input files with GICs from *\$gdvroot/gdv/tests/com* is given at the end of this document. The user is advised to look up those test files.

Redundant internal coordinates

Geom=Redundant or Opt=Redundant

Build an automatic set of redundant internal coordinates such as bonds, angles, and dihedrals from the current Cartesian coordinates or Z-matrix values using the old algorithm (*Peng96*) available in *Gaussian 16* (no GICs). If **Opt** is requested, perform the optimization using the Berny algorithm in these redundant internal coordinates. This is the default.

Geom=GIC or Opt=GIC

Build an automatic set of redundant internal coordinates using the new GIC algorithm. If **Opt** is requested, perform the optimization using the Berny algorithm with the GICs. The coordinates generated with this option can be the same bonds, angles, and dihedral angles generated using the old algorithm if the input geometry does not have any disjoint fragments that require a special treatment with the option **Geom=(GIC,Rotors)** or any methyl or similar XY₃-type functional groups attached to the main skeleton that require a special treatment with the option **Geom=(GIC,XY3)** (see below). With

a few exceptions noted below, the default is Geom=(GIC,Rotors,XY3).

Geom=AddGIC or Opt=AddGIC

Build the same GICs as described above. Then read the input stream for user-provided modifications to those GICs before performing any further calculation. This option requires a separate input section following the geometry specification. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. The options Geom=(GIC,ModRedundant) and Opt=(GIC,ModRedundant) are acceptable as well.

Geom=ReadAllGIC or Opt=ReadAllGIC

Do not build any redundant internal coordinates by default. Instead, read the input stream for user-provided GIC definitions and create the coordinates. This option requires a separate GIC input section following the geometry specification. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. The default is Geom=(ReadAllGIC,NoRotors,NoXY3).

Geom=GICOld or Opt=GICOld

Build an automatic set of redundant internal coordinates using the old algorithm (*Peng96*) and then convert the coordinates into the GICs and treat them as such. No other options such as Rotors or XY3 can be used here.

Z-matrix internal coordinates

Geom=GICZmat

Read regular Z-matrix coordinates from the input and then convert them into GIC-type internal coordinates with the original Z-matrix constraints being preserved. If Opt is requested, perform a geometry optimization like with Opt=Z-Matrix but in terms of the new GICs. The option Opt=GICZmat is acceptable as well. No other options such as Rotors or XY3 can be used here. To find the corresponding input examples, run the following command:
`grep -i 'GICZmat' $gdvroot/gdv/tests/com/*.com`

Geom=(GICZmat,AddGIC)

Build the GIC-type Z-matrix coordinates and constraints (as described above) and then read the input stream for user-provided modifications to those GICs before performing any further calculation. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. The option Opt=(GICZmat,AddGIC) is acceptable as well.

Model-specific internal coordinates

Geom=(GIC,Rotors)

Build a special set of GICs to treat disjoint fragments determined from the input Cartesian geometry or from the user-provided connectivity information and do not build any virtual bonds. This is the default for Geom=GIC if such fragments are detected and at least one fragment has more than two atoms, the total number of atoms in the system is under 500, and this is for a regular calculation that does not use periodic boundary conditions (PBCs), ONIOM with micro-iterations, QST2, QST3, Path, Cartesian atoms or the RdOpt option. Otherwise, the default for Geom=GIC is Geom=(GIC,NoRotors). The connectivity information can be introduced by using the option Geom=(Connectivity,GIC). The fragments can also be specified explicitly in the Cartesian input section as

O(fragment=1)	0.000000000000	0.000000000000	0.000000000000
H(fragment=1)	0.000000000000	0.000000000000	0.976756085872
N(fragment=2)	0.259655068263	0.000000000000	2.877762253635
H(fragment=2)	1.269044288026	0.000000000000	2.982267037494
H(fragment=2)	-0.089902960790	0.816744932713	3.367111592058
H(fragment=2)	-0.089902960790	-0.816744932713	3.367111592058
H(fragment=1)	-0.924602146888	0.000000000000	-0.254984428189

These GICs include the default internal coordinates (regular bonds, valence and dihedral angles) for individual fragments and an additional set of coordinates to describe the rotational and translational motion of each fragment. The translational GICs of fragment k with N_k atoms are the Cartesian coordinates of its center of geometry (labeled by default as C_{xk} , C_{yk} , and C_{zk}):

$$C_{xk} = (\sum X_{ik})/N_k$$

$$C_{yk} = (\sum Y_{ik})/N_k$$

$$C_{zk} = (\sum Z_{ik})/N_k$$

where $\sum$ means a sum over i and the index i runs over N_k atoms in the fragment defined by the atoms' Cartesian coordinates. The rotational GICs are the Cartesian components of the unit vectors P_k , Q_k , and S_k (labeled by default as P_{xk} , P_{yk} , P_{zk} , Q_{xk} , Q_{yk} , Q_{zk} , S_{xk} , S_{yk} , S_{zk}) to describe the three orthogonal axes of rotation. The vector P_k is directed from the center of geometry (C_k) towards the atom ($A_{k,1}$) most distant from the center (using the first qualified atom on the list if there are symmetrically equivalent ones) and it is defined by the three GICs:

$$P_{xk} = (X_{1k} - C_{xk})/R(1, C_k)$$

$$P_{yk} = (Y_{1k} - C_{yk})/R(1, C_k)$$

$$P_{zk} = (Z_{1k} - C_{zk})/R(1, C_k)$$

The direction of the unit vector Q_k is defined by a vector product of the vector P_k and the vector spanning from C_k to the second atom ($A_{k,2}$) (selected to give the smallest dot product $C_k A_{k,1} \cdot C_k A_{k,2}$ and not to be the atom closest to the center of the fragment at the same time):

$$Q_{xk} \sim P_{yk}(Z_{2k} - C_{zk}) - P_{zk}(Y_{2k} - C_{yk})$$

$$Q_{yk} \sim P_{zk}(X_{2k} - C_{xk}) - P_{xk}(Z_{2k} - C_{zk})$$

$$Q_{zk} \sim P_{xk}(Y_{2k} - C_{yk}) - P_{yk}(X_{2k} - C_{xk})$$

The vector S_k is the vector product $P_k \otimes Q_k$:

$$S_{xk} = P_{yk} Q_{zk} - P_{zk} Q_{yk}$$

$$S_{yk} = P_{zk} Q_{xk} - P_{xk} Q_{zk}$$

$$S_{zk} = P_{xk} Q_{yk} - P_{yk} Q_{xk}$$

An additional set of six GICs is used to freeze the total translations and rotations of the whole molecule with N atoms using Eckart conditions for translations and rotations:

$$(\sum X_k)/N \equiv 0$$

$$(\sum Y_k)/N \equiv 0$$

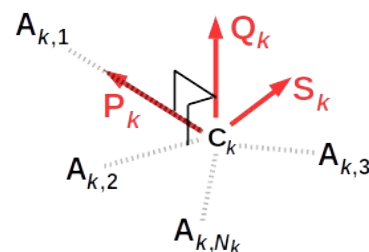
$$(\sum Z_k)/N \equiv 0$$

$$(\sum [Y_k^* Z_k - Z_k^* Y_k])/N \equiv 0$$

$$(\sum [Z_k^* X_k - X_k^* Z_k])/N \equiv 0$$

$$(\sum [X_k^* Y_k - Y_k^* X_k])/N \equiv 0$$

where $\sum$ means a sum over k which runs over all the N atoms in the molecule, and the asterisk refers to the initial values of Cartesian coordinates for atom k (to be determined automatically from the input geometry). Note that in certain calculations (e.g., for some non-regular cases such as PBCs) the translational and rotational coordinates of the largest fragment k (C_{xk} through S_{zk}) will be frozen by



default instead of using the aforementioned Eckart conditions. If the fragment is linear, the vectors Q_k and S_k are undefined and their components are replaced with the Cartesian coordinates of the remaining atoms (after the first one). If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(GIC,Rotors) is acceptable as well.

Geom=(AddGIC,Rotors)

Build a special set of GICs to treat disjoint fragments as described above and then read an additional GIC input to modify the new GICs. If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(AddGIC,Rotors) is acceptable as well. To find the corresponding input examples, run the following command:

```
grep -i Rotors $gdrvroot/gdv/tests/com/*.com | grep -v -i NoRotors
```

Geom=(ReadAllGIC,Rotors)

Read the input stream for user-provided GIC definitions and build a special set of rotor-type GICs to treat disjoint fragments as described above. The default is Geom=(ReadAllGIC,NoRotors). Note that the user-provided GIC definitions should include the internal coordinates for all the fragments of interest and also the actual fragment specifications if there are no default fragments found or the default fragments should not be used at all. If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(ReadAllGIC,Rotors) is acceptable as well.

Geom=(GIC,NoRotors) or Geom=(AddGIC,NoRotors) or Geom=(ReadAllGIC,NoRotors)

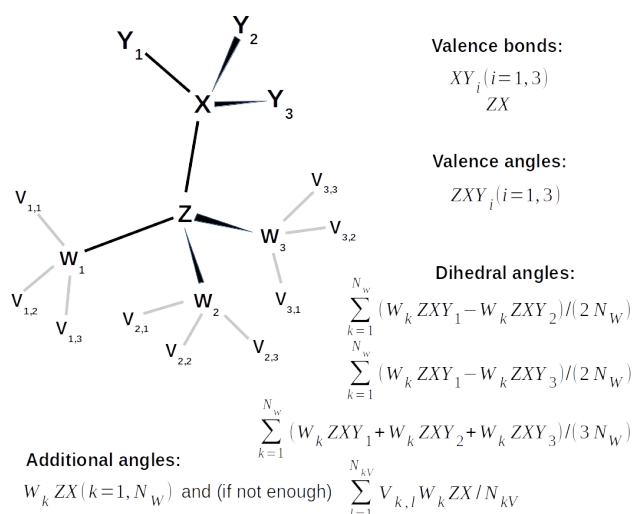
Do not build a special set of GICs to treat disjoint fragments even if there are such fragments. Use regular bonds, valence and dihedral angles as well as virtual bonds to connect the fragments, if any, or any coordinates that come from a user-provided GIC input section. Note that NoRotors is the default for Geom=GIC and AddGIC when used with PBCs, micro-iterative ONIOM, QST2 or similar options and it is always the default for Geom=ReadAllGIC.

Geom=(GIC,NoDefaultFrag) or Geom=(AddGIC,NoDefaultFrag) or Geom=(ReadAllGIC,NoDefaultFrag)

Do not look into the default or user-provided connectivity data for any default fragments. Use regular and virtual bonds, valence and dihedral angles or any coordinates that come from a user-provided GIC input section. Note that a combination of NoDefaultFrag and Rotors with no user-provided fragment information in the GIC input section will cause an error.

Geom=(GIC,XY3)

Identify any methyl (CH_3) or similar non-planar groups (XY_3 or $\text{XYY}'\text{Y}''$ in general) from the input geometry or connectivity. The connectivity information can be introduced by using the option Geom=(Connectivity,GIC). Build the default GICs (bonds, angles, and dihedral angles) for the remaining skeleton. Build a special set of GICs for the detected XY_3 -type groups (see the scheme on the right). This is the default for Geom=GIC for any molecule larger than neopentane (17 atoms) and a regular calculation without PBCs, ONIOM with



micro-iterations, QST2, QST3, Path, Cartesian atoms or the RdOpt option. Otherwise, the default for Geom=GIC is Geom=(GIC,NoXY3). If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(GIC,XY3) is acceptable as well.

Geom=(AddGIC,XY3)

Build a special set of GICs to treat the XY₃-type groups as described above and then read an additional GIC input to modify the new GICs. Note that such groups must be present if the XY3 option is specified explicitly. If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(AddGIC,XY3) is acceptable as well. To find the corresponding input examples, run the following command:

```
grep -i XY3 $gdvroot/gdv/tests/com/*.com | grep -i -v NoXY3
```

Geom=(ReadAllGIC,XY3)

Read the input stream for user-provided GIC definitions that should include the internal coordinates for the skeleton. Build a special set of GICs for the XY₃-type groups detected automatically. Note that such groups must be present. The default is Geom=(ReadAllGIC,NoXY3). If Opt is requested, perform a geometry optimization using the resulting GICs. The option Opt=(ReadAllGIC,XY3) is acceptable as well. To find the corresponding input examples, run the following command:

```
grep -i XY3 $gdvroot/gdv/tests/com/*.com | grep -i ReadAllGIC
```

Geom=(GIC,NoXY3) or Geom=(AddGIC,NoXY3) or Geom=(ReadAllGIC,NoXY3)

Do not build a special set of GICs to treat the XY₃-type groups even if there are such groups. Use regular bonds, valence and dihedral angles or any coordinates that come from a user-provided GIC input section. Note that NoXY3 is the default for Geom=GIC and AddGIC when used with PBCs, micro-iterative ONIOM, QST2 or similar options and it is always the default for Geom=ReadAllGIC.

Geom=GIC IOp(1/156=29)

Identify AB₃-type planar groups (or ABB'B" in general) and the corresponding near-planar groups where the sum of the three BAB angles exceeds 350°. Build a special set of umbrella-type GICs defined via the U(i,j,k,l) function described in *GIC SYMBOLIC INPUT DICTIONARY* (see a figure there) for the detected AB₃-type groups in addition to the initial set of GICs modified by removing the ABB'B" dihedral angles, if any. A set of GICs generated for an AB₃-type planar group with atoms A₁, B₂, B₃, and B₄ will be U(1,2,3,4), U(1,3,4,2), and U(1,4,2,3). AddGIC and ReadAllGIC can also be used. If Opt is requested, perform a geometry optimization using the resulting GICs. Find more examples by running the following command:

```
grep -i 'U(1' $gdvroot/gdv/tests/com/*.com
```

Symmetrized internal coordinates

Geom=GICAllSym

Build the default initial GICs as a set of regular bonds, valence and dihedral angles for the current input geometry and symmetrize them based on the current point group (determined from the input Cartesian coordinates) by building symmetrized linear combinations according to each irreducible representation. If Opt is requested, perform a geometry optimization using the symmetrized GICs for the totally symmetric irrep (e.g., for A_{1g} in O_h) with the remaining GICs being kept frozen to preserve the current symmetry. The option Opt=GICAllSym is acceptable as well. If the symmetry is not expected to change upon geometry optimization, GICSym (see below) should be used instead of GICAllSym. If the

options Opt=QST2 or Opt=QST3 are used as well then the symmetrization will be applied to the final GICs generated for the union (transition state) structure. If there are no other automatic options such as Rotors or XY3 specified explicitly then the initial (unsymmetrized) GICs will be regular internal coordinates with disjoint fragments being connected by virtual bonds (if any) and with no special treatment of XY₃-type functional groups (if any). The default is Geom=(GICAllSym,NoRotors,NoXY3).

Geom=GICSym or Opt=GICSym

Do the same as described above with respect to the totally symmetric irreducible representation only (e.g., for A_{1g} in O_h). If Opt is requested, perform a geometry optimization using the IOp(1/156=1xxx) settings and the GICs symmetrized only for the totally symmetric irrep. The option Opt=GICSym is acceptable as well. The default is Geom=(GICSym,NoRotors,NoXY3).

Geom=(GICAllSym,AddGIC) or Geom=(GICSym,AddGIC)

Build the default symmetrized GICs as described above. Then read any user-provided modifications to the final GICs from a GIC input section. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. The option ReadAllGIC can be used instead of AddGIC to symmetrize any user-provided set of GICs instead of the default GICs. To find the corresponding input examples, run the following command:

```
grep -i -e GICAllSym -e GICSym $gdrvroot/gdv/tests/com/*.com | grep -i ReadAllGIC
```

Note on QST2 and QST3 calculations with GIC, AddGIC, ReadAllGIC, and other GIC-related options

The QST2 calculation requires two geometry specifications as usual (one for the reactants and another for the products). The QST3 calculation requires three geometry specifications (for the reactants, for the products, and for the transition state). The options Opt=(QSTx,GIC) where x is equal to 2 or 3 do not require any additional GIC-related input. The options Opt=(QSTx,ReadAllGIC) require two separate GIC inputs as the only source of GICs where the first input follows the geometry specification for the reactants and the second input follows the geometry specification for the products (there is no GIC input for the third structure). The options Opt=(QSTx,AddGIC) require one GIC input after the last QSTx geometry specification to modify the final GICs generated by default for the union (transition state) geometry. The options Opt=(QSTx,AddGIC,Y) with Y to be replaced with another keyword such as GICSym, GICAllSym, Rotors, or XY3 (note that not every possible combination will be practical or has been tested so far) require the second GIC input to modify the final GICs obtained after an automatic modification (e.g., symmetrization) of the transition-state GICs. Any missing GIC input section that is otherwise required should be replaced with an extra blank line. Here is an example of the QST2 input when we need to modify the final set of GICs (by freezing something in this case):

```
# hf/3-21g opt=(QST2,AddGIC) nosymm
```

```
QST2 reactant
```

```
0 2
C      0.00000000  0.00000000  0.00000000
H      0.00000000  0.00000000  2.00000000
H      1.08000000  0.00000000  0.00000000
H     -0.54000000 -0.93530744  0.00000000
H     -0.54000000  0.93530744  0.00000000
F     -0.00000000  0.00000000  2.92000000
```

```
QST2 product
```

```

0 2
C      0.00000000  0.00000000  0.00000000
H      0.00000000  0.00000000  1.08000000
H      1.01817859  0.00000000 -0.36015602
H      -0.50908930 -0.88176853 -0.36015602
H      -0.50908930  0.88176853 -0.36015602
F      -0.00000000  0.00000000  3.08000000

```

L(1,2,6,3,-1) Frozen
L(1,2,6,3,-2) Frozen

Here is an input for the same transition state calculation using QST3:

```
# hf/3-21g opt=(QST3,AddGIC) nosymm
```

QST3 reactant

```

0 2
C      0.00000000  0.00000000  0.00000000
H      0.00000000  0.00000000  2.00000000
H      1.08000000  0.00000000  0.00000000
H      -0.54000000 -0.93530744  0.00000000
H      -0.54000000  0.93530744  0.00000000
F      -0.00000000  0.00000000  2.92000000

```

QST3 product

```

0 2
C      0.00000000  0.00000000  0.00000000
H      0.00000000  0.00000000  1.08000000
H      1.01817859  0.00000000 -0.36015602
H      -0.50908930 -0.88176853 -0.36015602
H      -0.50908930  0.88176853 -0.36015602
F      -0.00000000  0.00000000  3.08000000

```

QST3 transition state

```

0 2
C      0.00000000  0.00000000  0.00000000
H      0.00000000  0.00000000  1.25000000
H      1.06359237  0.00000000 -0.18754003
H      -0.53179619 -0.92109801 -0.18754003
H      -0.53179619  0.92109801 -0.18754003
F      -0.00000000  0.00000000  2.50000000

```

L(1,2,6,3,-1) Frozen
L(1,2,6,3,-2) Frozen

To find more input examples, run the following command:

```
grep -i 'QST' $gdvroot/gdv/tests/com/*.com | grep -i GIC
```

Check-point reading options

Geom=Check

Read the Cartesian geometry and any internal coordinate definitions (GICs or others) from the check-point file. To find some input examples that use Geom=Check combined with various GIC options (e.g., GICAllSym), run the following command:

```
grep -i 'GIC' $gdvroot/gdv/tests/com/*.com | grep -i check
```

Geom=(Check,GIC) or Geom=(Check,GIC,NewDefinition)

Read the Cartesian geometry and discard any current internal coordinate definitions (GICs or others) from the check-point file. Build a new set of the default GICs for the current geometry. The option Geom=(Check,NewDefinition) is equivalent to Geom=(Check,Redundant,NewDefinition). No options such as Rotors and XY3 will be applied by default.

Geom=(Check,AddGIC) or Geom=(Check,GIC,ModRedundant)

Read the Cartesian geometry and current GIC definitions from the check-point file. Then read the input stream for user-provided modifications to the current GICs before performing any further calculation. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. No options such as Rotors and XY3 will be applied by default.

Geom=(Check,ReadAllGIC)

Read the Cartesian geometry and discard any current internal coordinate definitions (GICs or others) from the check-point file. Read the input stream for user-provided GIC definitions. The syntax of the GIC input section is described in *GIC SYMBOLIC INPUT DICTIONARY*. No options such as Rotors and XY3 will be applied by default.

GIC SYMBOLIC INPUT DICTIONARY

General information

The GIC input section is separated from the earlier input by a blank line. It has one or more lines containing coordinate definitions, expressions or standalone options. Here is a simple GIC input section for water illustrating some of the possible features:

R(1,2)	! Define a bond coordinate between atom 1 and atom 2
Bond2=R[1,3]	! Define another bond and name it Bond2
HOH(freeze)=A(2,1,3)	! Define the valence angle and keep it frozen during geometry optimization

For an optimization, these coordinates will result in the bond angle remaining fixed at its initial value and the two bond distances being optimized. Note that the text after the ! mark is just an optional comment.

The basic form of a coordinate definition is the following:

$$\text{Label}(\text{Options})=\text{Expression}$$

All of the components are optional. In the preceding examples, all components were present only in the third line. The first line contained only a coordinate expression, while the second line also contained a label without options. Note that options may also be placed following the expression without a label:

$$\text{Expression Options}$$

Note that the line “A(2,1,3) Freeze” will be valid while the line “HOH=A(2,1,3) Freeze” will not.

Labels are user-assigned identifiers for the coordinates. They are not case sensitive. Labels may contain letters and numbers, but must begin with a letter. If no label is specified, a generic one will be assigned by the program. The default label for bonds will be R1, R2, ..., Rn where *n* runs over all generated bonds (active, frozen, and inactive) in the order of appearance. In a similar fashion, the default labels for angles will be A1, A2, etc. and default labels for dihedral angles will be D1, D2, etc. The default label for a generic GIC without a user-provided label is Gi where *i* is the index of the corresponding GIC primitive in the whole GIC expression tree. The latter can be printed out using IOp(1/33=3), if necessary for debugging (note that the log file may be very large). A parenthesized, comma-separated

list of options can be included following the label, if desired.

Coordinates are defined by expressions. The simplest expressions identify a specific structural parameter such as a bond, for example, R(1,2) where the indexes refer to specific atoms. Note that an asterisk may be used as a wildcard for any atom number in certain situations and the dummy atoms specified by the dummy() function can also be used. A coordinate definition expression in general may include available mathematical functions (arccos, sqrt, etc.) and operations (“+”, “-”, “*”, “/”, “**2”, etc.), as well as the labels of previously defined coordinates (R1, R2, etc.). The expression may use parentheses, square brackets, or braces. All of them can be nested but mixing the different types, e.g. {}, is not permitted. The letters in the expression are not case sensitive. The expression should not begin with “+”, “*”, “/” but may begin with “-”. For example, the line “-R1+R2” is permitted and will be equivalent to “R2-R1”, and expressions such as “(-1.)*R1+R2”, “-1.*R1+R2”, “-2.*R(1,2)”, and “-(-R(1,2)+R(1,3))” are all valid.

List of available functions to specify geometry parameters

R(i,j)

Define a bond coordinate between atoms *i* and *j*. Synonyms: B(i,j), Bond(i,j), Stretch(i,j).

A(i,j,k)

Define a non-linear angle coordinate involving atoms *i*, *j*, and *k* where atom *j* is at the angle vertex. Synonyms: Angle(i,j,k), Bend(i,j,k).

D(i,j,k,l)

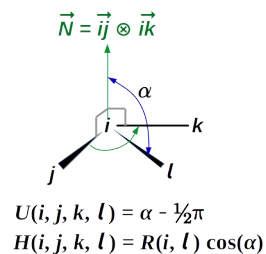
Define a dihedral angle between the plane containing atoms *i*, *j*, and *k* and the plane containing atoms *j*, *k*, and *l*. Synonyms: Dihedral(i,j,k,l), Torsion(i,j,k,l).

L(i,j,k,L,M)

Define the first ($M = -1$) or the second ($M = -2$) component of the linear bend coordinate involving atoms *i*, *j*, and *k* where atom *j* is at the angle vertex. The values of *M* other than -1 or -2 are not permitted. The *L* can be -4, -3, -2, -1, 0, or >0. A value of $L = -4$ or $L > 0$ is applicable only to a nonlinear molecule with more than three atoms. A value of $L > 0$ means that the linear bend between *i*, *j*, and *k* is specified using the fourth atom *L* which does not form a linear angle with *i*, *j*, and *k* in any combination. The fourth atom is used to build the two orthogonal directions of the linear bend (one for $M = -1$ and another for $M = -2$). If $L = -4$, the fourth atom will be determined by default based on the molecule's geometry. The values $L = 0, -1, -2$, or -3 mean that the fourth atom is not used and the given component *M* is determined by projecting the linear bend onto the YZ, XZ, or XY plane in the case of $L = -1, -2$, or -3 , respectively, or onto any of these planes as determined by default if $L = 0$. The 3-atom linear bend is the only option for a linear molecule. Synonyms: Linear(i,j,k,L,M), LinearBend(i,j,k,L,M).

U(i,j,k,l)

Define an umbrella-type coordinate involving atoms *i*, *j*, *k*, and *l* as an angle formed by the *i-l* vector and the *i-N* vector minus 90° where *i-N* is the vector product of *i-j* and *i-k*. See the figure on the right. The coordinate changes from -90° to 90°.



H(i,j,k,l)

Define an umbrella-type coordinate involving atoms i , j , k , and l as a signed distance between the j - i - k plane and the atom l . See the same figure. The coordinate changes from $-R(i,l)$ to $R(i,l)$ where $R(i,l)$ is the distance from i to l .

X(i)

Define Cartesian coordinate X for atom i . Synonyms: $\text{Cart}(i,-1)$, $\text{Cart}(i,X)$, $\text{Cartesian}(i,-1)$, $\text{Cartesian}(i,X)$.

Y(i)

Define Cartesian coordinate Y for atom i . Synonyms: $\text{Cart}(i,-2)$, $\text{Cart}(i,Y)$, $\text{Cartesian}(i,-2)$, $\text{Cartesian}(i,Y)$.

Z(i)

Define Cartesian coordinate Z for atom i . Synonyms: $\text{Cart}(i,-3)$, $\text{Cart}(i,Z)$, $\text{Cartesian}(i,-3)$, $\text{Cartesian}(i,Z)$. The letters X , Y , Z are not case sensitive.

DotDiff(i,j,k,l)

Define the dot product ($a \cdot b$) of the two Cartesian coordinate difference vectors a and b for atoms i , j , k , and l determined as $a = (X_i - X_j, Y_i - Y_j, Z_i - Z_j)$ and $b = (X_k - X_l, Y_k - Y_l, Z_k - Z_l)$. The values i , j , k , and l can be equal to any valid atomic center number.

Label=Dummy(X,Y,Z)

Define a GIC dummy atom along with some user-provided label (replace "Label" with the actual label) whose Cartesian coordinates are either functions of some previously defined GICs (X , Y , and Z should be replaced with actual labels for those GICs) or simply some points in space where X , Y , and Z are floating-point values in the units defined by the Units keyword (Angstroms by default). This GIC does not add any new coordinate and it is used as an intermediate expression to build other GICs. Find some examples of using GIC dummy atoms by running the following command:

```
grep -i 'Dummy(' $gdrvroot/gdv/tests/com/*.com
```

Fragment(i-j)**Label=Fragment(i-j)**

Define a molecular fragment along with an optional label (replace "Label" with the actual label) that contains specified atoms separated by commas or/and hyphens. The default labels are FRGM1, FRGM2, etc. For example, the interval (1,2-5,7) refers to atoms #1, 2, 3, 4, 5, and 7. Wildcards (asterisks) and dummy atoms are not permitted. To specify a fragment that extends to the whole molecule (for example, with 8 atoms) use the interval (1-8). This GIC does not add any new coordinate and it is used as an intermediate expression to build other GICs (see examples).

Label(Frozen)=Fragment(i-j)

Define a molecular fragment as described above and freeze the corresponding atoms by adding their frozen Cartesian coordinates explicitly to the current internal coordinates. This is similar to using the RdOpt option along with the NotAtoms list.

XCntr(i-j)**YCntr(i-j)**

ZCntr(i-j)

Define Cartesian coordinates ($X_{\text{Cntr}}, Y_{\text{Cntr}}, Z_{\text{Cntr}}$) for the geometric center (centroid) of a molecular fragment that contains the atoms from the interval (i - j) used to specify a fragment. The interval can be replaced with a label for that fragment. The functions are defined as $X_{\text{Cntr}} = (\sum X_k)/N$, $Y_{\text{Cntr}} = (\sum Y_k)/N$, and $Z_{\text{Cntr}} = (\sum Z_k)/N$ where $\sum$ is a sum over k which runs over all the N atoms in the specified interval. The functions can be given with a user-provided label and some action flag, e.g., as $X0(\text{Frozen})=XCntr(1-5)$. These GICs can be frozen during geometry optimization to constrain the fragment's total translation if some of the remaining coordinates describing the fragment are not translationally invariant.

X2Cntr(i-j)

Y2Cntr(i-j)

Z2Cntr(i-j)

Define the expressions $X2_{\text{cctr}} = (\sum Y_k Z_k)/N$, $Y2_{\text{cctr}} = (\sum X_k Z_k)/N$, and $Z2_{\text{cctr}} = (\sum X_k Y_k)/N$ where $\sum$ is a sum over k which runs over all the N atoms from the interval (i - j) used to specify a fragment. The interval can be replaced with a label for that fragment. The GICs correspond to the off-diagonal elements of the moment of inertia tensor built over N points (all masses set to 1) and they are zero if the fragment's Cartesian coordinates correspond to the standard orientation. These GICs can be frozen during geometry optimization to constrain the fragment's total rotation if some of the remaining coordinates describing the fragment are not rotationally invariant.

XEckart(i-j)

YEckart(i-j)

ZEckart(i-j)

Define the expressions $X_{\text{Eckart}} = (\sum [Y_k^* Z_k - Z_k^* Y_k])/N$, $Y_{\text{Eckart}} = (\sum [Z_k^* X_k - X_k^* Z_k])/N$, and $Z_{\text{Eckart}} = (\sum [X_k^* Y_k - Y_k^* X_k])/N$ where $\sum$ is a sum over k which runs over all the N atoms from the interval (i - j) used to specify a fragment, and X_k^* , Y_k^* , and Z_k^* refer to the initial values of Cartesian coordinates for atom k (from the input file). The interval can be replaced with a label for that fragment. The GICs represent the rotational Eckart conditions and can be frozen during geometry optimization to constrain the fragment's total rotation if some of the remaining coordinates describing the fragment are not rotationally invariant. Note that the translational Eckart conditions can be satisfied by freezing the corresponding translational GICs ($X_{\text{Cntr}}, Y_{\text{Cntr}}, Z_{\text{Cntr}}$) introduced above.

Rotor(1)

For a molecular system with two or more fragments, replace the current set of bonds, valence and dihedral angles that describe the relative motion of the fragments with a special set of rotational and translational GICs for each fragment ($Cxk, Cyk, Czk, P_xk, P_yk, P_zk, Q_xk, Q_yk, Q_zk, S_xk, S_yk$, and S_zk , where k runs over all fragments), retain the internal coordinates of individual fragments, and add the rotational and translational constraints for the whole molecule. It is intended for use with `Geom=AddGIC` and it is identical to using "`Geom=(GIC,Rotors)`" described above. Therefore, the two inputs below will do the same calculation (note that the coordinate labels may differ in these two cases):

```
#p UFF Geom=(AddGIC,NoRotors)
```

```
Input 1
```

```
0 1
O -1.089969 0.309810 -0.194332
H -1.632065 0.635159 0.561017
H -0.261990 0.845646 -0.092644
O 1.198474 1.585153 0.139746
```

```

H  1.644173  1.799808 -0.713347
H  1.840674  0.951954  0.539024
O  -0.263009 -2.180493  0.178888
H  -0.824725 -2.686015 -0.457260
H  -0.611562 -1.261022  0.038907

```

Rotor(1)

--Link1--

#p UFF Geom=(Gic,Rotors)

Input 2

0 1

```

O  -1.089969  0.309810 -0.194332
H  -1.632065  0.635159  0.561017
H  -0.261990  0.845646 -0.092644
O  1.198474  1.585153  0.139746
H  1.644173  1.799808 -0.713347
H  1.840674  0.951954  0.539024
O  -0.263009 -2.180493  0.178888
H  -0.824725 -2.686015 -0.457260
H  -0.611562 -1.261022  0.038907

```

Note that the example above uses the fragments determined by default based on the input Cartesian coordinates. However, the user can specify the fragments explicitly and can also specify other modifications before and after executing the Rotor(1) statement. In the two examples below (they are again identical), one of the coordinates built by Rotor(1) is frozen.

#p UFF Geom=(AddGIC,NoDefaultFrag)

Input 3

0 1

```

O  -1.089969  0.309810 -0.194332
H  -1.632065  0.635159  0.561017
H  -0.261990  0.845646 -0.092644
O  1.198474  1.585153  0.139746
H  1.644173  1.799808 -0.713347
H  1.840674  0.951954  0.539024
O  -0.263009 -2.180493  0.178888
H  -0.824725 -2.686015 -0.457260
H  -0.611562 -1.261022  0.038907

```

Fragment(1-3)

Fragment(4-6)

Fragment(7-9)

Rotor(1)

Cx1 Freeze

--Link1--

#p UFF Geom=(AddGIC,Rotors)

Input 4

0 1

```

O  -1.089969  0.309810 -0.194332
H  -1.632065  0.635159  0.561017
H  -0.261990  0.845646 -0.092644
O  1.198474  1.585153  0.139746
H  1.644173  1.799808 -0.713347
H  1.840674  0.951954  0.539024
O  -0.263009 -2.180493  0.178888
H  -0.824725 -2.686015 -0.457260
H  -0.611562 -1.261022  0.038907

```

Cx1 Freeze

Note that “1” in “Rotor(1)” is just a model option, and it is the only option that is currently available. Note that ReadAllGIC can be used instead of AddGIC but in this case the user should provide the internal coordinates for each fragment without any inter-fragment GICs. The Rotor(1) option will build the latter automatically.

Rotor(Label)

Replace the current GICs that describe the motion of the given fragment (note the label should start with a character) relative to the remaining fragments with a special set of rotational and translational GICs for that fragment. The user should specify the fragments explicitly along with their labels using the `Fragment()` specification prior to executing this option. If there are no user-provided fragments then the program will attempt to assign the default fragments (labeled by default as FRGM1, FRGM2, etc.) using the current connectivity data based on the input Cartesian geometry, if possible. The user can freeze the added rotational and translational GICs of the given fragment as shown in the example below (note that the corresponding intra-fragment internal coordinates will not be frozen by doing so):

```
#p UFF Geom=(AddGIC,NoRotors)
```

```
Input 5
```

```
0 1
O      1.87186095  1.25047332  0.02579685
H      2.73702835  0.83491096  0.04562845
H      1.97522949  2.20358349 -0.02416271
O      3.06648364 -1.18456722 -0.06585758
H      3.43660780 -1.25180794  0.81736763
H      3.42642471 -0.41112706 -0.50612198
O     -5.20505830 -0.24403561  0.00179024
H     -4.33989090 -0.65959797  0.02162183
H     -5.10168976  0.70907456 -0.04816932
```

```
Wat1=Fragment(1-3)
```

```
Wat2=Fragment(4-6)
```

```
Wat3=Fragment(7-9)
```

```
!
```

```
! Build the new GICs for each fragment one by one and freeze the first one
```

```
RWat1(Frozen)=Rotor(Wat1)
```

```
Rotor(Wat2)
```

```
Rotor(Wat3)
```

Note the option `NoRotors` added to the input above to avoid building the default rotor-type GICs for the existing fragments because they would conflict with the user-provided GICs.

Rotor(Label1,Label2)

Replace the current GICs that describe the motion of the fragments `Label1` and `Label2` relative to each other and the remaining fragments with a special set of rotational and translational GICs (see above). Then express them in terms of dot products:

$\Delta C_1 \cdot P_2$, $\Delta C_1 \cdot Q_2$, $\Delta C_1 \cdot S_2$,

$\Delta C_2 \cdot P_1$, $\Delta C_2 \cdot Q_1$, $\Delta C_2 \cdot S_1$,

$P_1 \cdot P_2$, $Q_1 \cdot P_2$, $S_1 \cdot P_2$,

$P_1 \cdot Q_2$, $Q_1 \cdot Q_2$, $S_1 \cdot Q_2$,

$P_1 \cdot S_2$, $Q_1 \cdot S_2$, $S_1 \cdot S_2$

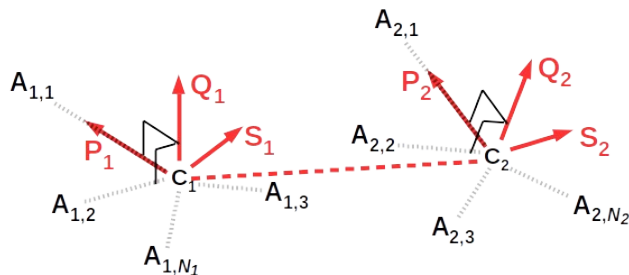
where $\Delta C_1 = C_1 - C_2$, $\Delta C_2 = C_2 - C_1$,

$C_k = (C_{xk}, C_{yk}, C_{zk})$, $P_k = (P_{xk}, P_{yk}, P_{zk})$,

$Q_k = (Q_{xk}, Q_{yk}, Q_{zk})$, $S_k = (S_{xk}, S_{yk}, S_{zk})$, and $k = 1$

refers to the fragment `Label1` and $k = 2$ refers to the fragment `Label2`. The vector C_k defines the position of the geometric center of the corresponding fragment with respect to the origin of Cartesian coordinates. The user should specify the fragments explicitly along with their labels using the `Fragment()` specification prior to executing this option. If there are no user-provided fragments then the program will attempt to assign the default fragments (labeled by default as FRGM1, FRGM2, etc.) using the current connectivity data based on the input Cartesian geometry, if possible. To find some input examples that use this option, run the following command:

```
grep -i 'Rotor(' $gdvroot/gdv/tests/com/*.com | grep -e ' '
```



List of available mathematical operations

Unary operations

The available unary operations described below operate on an argument A which can be the label of a previously defined coordinate or a valid GIC symbolic expression including an integer or floating-point number.

SQRT(A)

Take the square root of an argument.

EXP(A)

Take the exponent of an argument, i.e., e^A .

SIN(A)

Take the sine of an argument.

COS(A)

Take the cosine of an argument.

TAN(A)

Take the tangent of an argument.

ARCCOS(A)

Take the inverse cosine of an argument or arc-cosine.

Binary operations

The available binary operations described below operate on the arguments A and B which can be the labels of previously defined coordinates or any valid GIC symbolic expressions that involve integer or floating-point numbers.

A+B

Addition of two arguments.

A-B

Subtraction of two arguments.

A*B

Multiplication of two arguments.

A/B

Division of argument A by argument B .

Other operations

AN**

Take the N -th power of an argument. The base A can be the label of a previously defined coordinate or a valid GIC symbolic expression. The exponent N is an integer. Synonyms: A^N .

Options given with the coordinate's label or expression

The word “Options” refers to one or more GIC-related keywords separated by commas in the case of “Label(Options)” format or by spaces in the case of the “Expression Options” format. In particular, one of the keywords can be the coordinate's action flag having one of the following (case-insensitive) values.

Active

Build a new coordinate defined by the given expression with an optional user-provided label and flag it as active and unfrozen (i.e., add it to the list of internal coordinates used in geometry optimization and let its value change during the optimization). If the given expression or the coordinate's label has been previously added as inactive or frozen, then flag the coordinate as active and unfrozen. Note that this option is similar to the option A used in the ModRedundant input section with `Geom=ModRedundant` for the old algorithm (Peng96). Synonyms: Activate, Add. This flag is default.

Frozen

Build a coordinate defined by the expression (with an optional user-provided label) if it does not exist, flag the coordinate active for geometry optimization and freeze it at the current value. Note that this option is similar to the option F used in the ModRedundant input section with `Geom=ModRedundant` for the old algorithm (Peng96). Synonyms: Freeze.

Remove

(1) Build a new coordinate defined by the expression (with an optional user-provided label) and flag it inactive (i.e., do not include it in geometry optimization), or (2) if the coordinate with the given label or for the given expression has been already built and flagged as active frozen or active unfrozen then remove it from the geometry optimization by flagging it as inactive. Note that this option is similar to the option R used in the ModRedundant input section with `Geom=ModRedundant` for the old algorithm (Peng96). Synonyms: Inactive.

RemoveAll

Remove the coordinate from the list of internal coordinates used in geometry optimization along with any dependent coordinates by flagging all of them as inactive. The dependent coordinates include any coordinate that depends on the same atoms as the given coordinate. For example, “R(1,5) RemoveAll” will result in removing the coordinate R(1,5) referring to the internuclear distance between atom 1 and atom 5 as well as the valence angles, dihedral angles, or any arbitrary coordinate that depend on the Cartesian coordinates of atoms 1 and 5 in combination with any other atoms in the molecule. Note that this option is similar to the option K used in the ModRedundant input section with `Geom=ModRedundant` for the old algorithm (Peng96). Synonyms: Kill.

PrintOnly

Print an initial value of the coordinate defined by the expression or by the label along with the other

active coordinates and then remove it (i.e., flag as inactive) from the list of internal coordinates used in geometry optimization.

Modify

In the case of the “*Label(Options)=Expression*” format only, replace the old coordinate known by the label “*Label*” with the new expression and flag the newly modified coordinate as active and unfrozen.

Diff

Calculate numerical second derivatives for the row and column of the initial Hessian for this coordinate. Note that this option is similar to the option D used in the ModRedundant input section with Geom=ModRedundant for the old algorithm (Peng96).

Note that in the case of the “*Expression Options*” format, the flags Active, Diff, Frozen, Remove, and RemoveAll can be replaced with a single letter A, D, F, R, and K, respectively.

FC=X

Change the diagonal element for the coordinate in the initial Hessian to *X* where *X* is a floating-point value in atomic units. Note that this option is similar to the option H used in the ModRedundant input section with Geom=ModRedundant for the old algorithm (Peng96). Synonyms: ForceConstant.

Value=X

Set an initial value for the given internal coordinate to *X* where *X* is a floating-point value in the units defined by the Units keyword (Angstroms or degrees by default). The current Cartesian coordinates will be adjusted to match this value as closely as possible. This option should be used cautiously and sparingly. It is far easier and more reliable to set the initial molecular structure as desired in a graphic environment like GaussView.

StepSize=X,NSteps=N

Perform a relaxed potential energy surface scan. Increment the coordinate *N* times by the step *X* where *X* is a floating-point value in the units defined by the Units keyword (Angstroms or degrees by default). In the case of the “*Expression Options*” format, the comma between the keywords can be replaced with a space.

Min=XMn,Max=XMx

Add, freeze, or remove the coordinate if during the geometry optimization its value *X* satisfies the $XMn \leq X \leq XMx$ relation where *XMn* and *XMx* are user-provided floating-point values in the units defined by the Units keyword (Angstroms or degrees by default). If there is no Min=*XMn* given, the condition reduces to $X \leq XMx$. If there is no Max=*XMx* given, the condition reduces to $XMn \leq X$. The Min/Max option can only be used in combination with the flags Active, Frozen, or Remove. In the case of the “*Expression Options*” format, the comma between the keywords can be replaced with a space. Some examples of using these options can be found by running the following command:
`grep -i -e 'Min=' -e 'Max=' $gdvroot/gdv/tests/com/*.com`

There are two more keywords that can be used along with the coordinate’s expression.

Expression Flag OnlyIf Expression2

Add, freeze, or remove a coordinate defined by “*Expression*” only if another (previously introduced)

coordinate defined by “*Expression2*” is included in the geometry optimization. The word “*Flag*” should be replaced with the flags Active, Frozen, or Remove to add, freeze, or remove, respectively. The words “*Expression*” and “*Expression2*” should be replaced with mathematical expressions (without the “=” signs) or with the coordinates’ labels if the latter are known. The keyword OnlyIf is not case sensitive but should be spelled as one word.

Expression Flag IfNot Expression2

Add, freeze, or remove a coordinate defined by “*Expression*” only if another (previously introduced) coordinate defined by “*Expression2*” is *not* included in the geometry optimization. The word “*Flag*” should be replaced with the flags Active, Frozen, or Remove (or with their synonyms) to add, freeze, or remove, respectively. The words “*Expression*” and “*Expression2*” should be replaced with mathematical expressions (without the “=” signs) or with the coordinates’ labels if the latter are known. The keyword IfNot is not case sensitive but should be spelled as one word.

Standalone options

There are several options that can be used without the coordinate’s label or expression and they serve as global instructions.

FreezeAll

Freeze any internal coordinate previously added as active to the list of coordinates used in geometry optimization.

UnFreezeAll

Unfreeze any internal coordinate previously added as active frozen to the list of coordinates used in geometry optimization.

RemoveAll

Remove any internal coordinate previously added as active frozen and active unfrozen from the list of coordinates used in geometry optimization.

Atom *i*

Add the Cartesian coordinates of atom *i* to the list of internal coordinates used for geometry optimization. If the integer *i* is replaced with the wildcard symbol “*” then add the Cartesian coordinates of all atoms.

Atom *i* Freeze

Add the Cartesian coordinates of atom *i* to the list of internal coordinates used for geometry optimization and freeze them at their initial values.

Atom *i* UnFreeze

Unfreeze the Cartesian coordinates of atom *i* previously added to the list of internal coordinates as frozen.

Atom *i* Remove

Remove the previously added Cartesian coordinates of atom *i* from the list of internal coordinates used for geometry optimization.

Atom *i* RemoveAll

Remove the Cartesian coordinates of atom *i* from the list of internal coordinates used for geometry optimization and also remove any other internal coordinate that depends on atom *i*.

Atom *i* XYZOnly

Remove any internal coordinate that depends on atom *i* from the list of coordinates used for geometry optimization but add or keep the Cartesian coordinates of that atom.

RdOpt

There may be situations when certain atoms need to be frozen in terms of their Cartesian coordinates while keeping the internal coordinates connecting those atoms to the remaining ones unfrozen. Here is an example how to proceed in such a case:

```
#p HF/STO-3G Opt=(RdOpt,GIC)
```

```
Test 1
```

```
0 1
C      -0.08857879  0.12689103  0.00000000
O      -0.62172704  1.20626440  0.00000000
O      -0.79640759 -1.03149748  0.00000000
H      -1.73495142 -0.78582182  0.00000000
C      1.39215704  -0.12787100  0.00000000
H      1.56703747  -0.91337923  0.89935717
H      1.56703747  -0.91337923  -0.89935717
H      1.92446969  0.81936763  0.00000000
```

```
notatoms=1-4
```

In this example, the default GICs over all the atoms are modified by replacing the internal coordinates interconnecting the atoms 1, 2, 3, and 4 (which need to be frozen) with the frozen Cartesian coordinates of those atoms while keeping the remaining GICs as is. The latter include the GICs over the remaining (unfrozen) atoms and the GICs connecting the frozen atoms to the unfrozen ones. Here is another way to freeze the atoms:

```
#p HF/STO-3G Opt=GIC
```

```
Test 1
```

```
0 1
C  -1  -0.08857879  0.12689103  0.00000000
O  -1  -0.62172704  1.20626440  0.00000000
O  -1  -0.79640759 -1.03149748  0.00000000
H  -1  -1.73495142 -0.78582182  0.00000000
C      1.39215704  -0.12787100  0.00000000
H      1.56703747  -0.91337923  0.89935717
H      1.56703747  -0.91337923  -0.89935717
H      1.92446969  0.81936763  0.00000000
```

Use of the old ModRedundant format

Modifications to the GICs can be read in by using the ModRedundant format from the old internal coordinate algorithm available in *Gaussian 16*. See the ModRedundant section in the *G16* reference for the Opt keyword. However, the old format is only available with the GICs that include only pure bonds, angles, or torsions. In addition, the old format and the new GIC format described above cannot be mixed together in the same input section.

Note about the GIC units in Gaussian output

The values of the GICs defined as pure distances (labeled as “R”), valence angles or linear bends (“A”), and dihedral angles or torsions (“D”) are computed from the Cartesian coordinates in atomic units (Bohrs) and stored internally in Bohrs and radians (respectively, for distances and angles) but, for the user’s convenience, they are printed out in Angstroms (Å) and degrees (see, for example, the “Initial Parameters” table in the *Gaussian* output from any calculation that uses such GICs). In the case of a generic GIC (i.e., when the GIC is not a pure Cartesian coordinate, a bond distance, or an angle), the GIC value is computed as a function of Cartesian coordinates in Bohrs, bond distances in Bohrs, and/or valence, linear or dihedral angles in radians combined with optional constants (e.g., weights or scale factors) in user-defined units. Such generic GIC values (labeled as “GIC”) are computed, stored and printed in the same units which are procedure defined or arbitrary units in general. If the GIC is a combination of bonds or a combination of valence angles scaled with dimensionless weight factors then the arbitrary units become Bohrs for the bonds and radians for the angles.

Further examples of using GICs

Basic GIC input. Here is an example of using the generalized internal coordinates defined by the user from scratch for the geometry optimization of the water molecule.

```
# HF opt=ReadAllGIC

Title

0 1
O 0.0000 0.0000 0.0000
H 0.0000 0.0000 1.3112
H 1.0354 0.0000 -0.6225

R(1,2)
R(1,3)
HOH=A(2,1,3)
```

The atomic indexes 1, 2, and 3 refer to the oxygen atom, the first and the second hydrogen atom, respectively. The first and the second expression define the O-H bonds, and the third one defines the H-O-H valence angle (with the user-provided label “HOH”). An excerpt of the output with a table containing the initial values of the GICs is shown below.

! Initial Parameters ! ! (Angstroms and Degrees) !				
! Name	Definition	Value	Derivative Info.	!
! R1	R(1,2)	1.3112	estimate D2E/DX2	!
! R2	R(1,3)	1.2081	estimate D2E/DX2	!
! HOH	A(2,1,3)	121.015	estimate D2E/DX2	!

Note that the labels “R1” and “R2” above were assigned by default. The coordinates R1=R(1,2) and R2=R(1,3) are parsed as pure distances and given here in Angstroms, and the HOH=A(2,1,3) is a pure valence angle in degrees.

```
# HF opt=ReadAllGIC

Title

0 1
O 0.0000 0.0000 0.0000
H 0.0000 0.0000 1.3112
H 1.0354 0.0000 -0.6225
```

```
OHSym1=(R(1,2)+R(1,3))/sqrt(2)
OHSym2=(R(1,2)-R(1,3))/sqrt(2)
HOH=A(2,1,3)
```

The first and the second expression in the example above define the symmetrized O-H bonds, and the third one is the H-O-H valence angle.

! Initial Parameters ! ! (Angstroms and Degrees) !			
! Name	Definition	Value	Derivative Info.
! OHSym1	GIC-9	3.3664	estimate D2E/DX2
! OHSym2	GIC-11	0.1377	estimate D2E/DX2
! HOH	A(2,1,3)	121.015	estimate D2E/DX2

NOTE: GIC-type coordinates are in arbitrary units.

The coordinates OHSym1 and OHSym2 are parsed as generic GICs and therefore given here in arbitrary units. The units are actually Bohrs in this case because the $2^{-1/2}$ factor is taken as dimensionless and the values of R(1,2) and R(1,3) are taken in Bohrs.

```
# HF opt=ReadAllGIC
```

```
Title
```

```
O 1
O
H 1 1.3
H 1 1.2 2 120.
```

```
R12=SQRT[{X(2)-X(1)}^2+{Y(2)-Y(1)}^2+{Z(2)-Z(1)}^2]
R13=SQRT[{X(3)-X(1)}^2+{Y(3)-Y(1)}^2+{Z(3)-Z(1)}^2]
A0(Inactive)=DotDiff(2,1,3,1)/(R12*R13)
A213=ArcCos(A0)
```

The GIC input section above defines two bond distances and one valence angle expressed via Cartesian coordinates. The coordinate A0 is defined as the dot product (DotDiff) of the vectors and divided by the product of their lengths, and it is selected as “inactive” (i.e., excluded from the geometry optimization). An excerpt of the output with a table containing the initial values of the GICs is shown below.

! Initial Parameters ! ! (Angstroms and Degrees) !			
! Name	Definition	Value	Derivative Info.
! R12	GIC-15	2.4566	estimate D2E/DX2
! R13	GIC-27	2.2677	estimate D2E/DX2
! A213	GIC-31	2.0944	estimate D2E/DX2

NOTE: GIC-type coordinates are in arbitrary units.

The values of R12, R13, and the dot product are calculated using the Cartesian coordinates given in Bohrs. The GIC arbitrary units are Bohrs (for R12 and R13) and radians (for A213).

The next section describes the GIC-related IOps that can be used for debugging and for testing some experimental features. See, for example, “Test Job 1358” with IOp(1/145+=100000) that can be used to optimize the number of primitive GICs (which include the internal coordinates, their derivatives, and any intermediate expressions). Such GIC expression optimization may help to reduce the time and memory requirements to build a set of GICs with too many atoms and too many generic GICs (i.e., anything other than pure bonds and angles).

GIC-RELATED IOPS

Overlay 1

```
C      IOp(145) Controls for the new internal coordinate data structure.
C      0 ... Should the new internal coordinate data structure be set-up
C            for use? (Default=1).
C      1 ... No.
C      2 ... Yes. The coordinates are either generated by default (e.g.,
C            Option 9x2) or obtained from the user-provided input (e.g.,
C            Option 6x2). Alternatively, they can be taken from a chk file
C            if they exist there and if geom=check without any GIC keyword
C            (i.e., IOp(1/145=0) is used.
C      3 ... Yes, but do two calls to CrdDef when building the coordinate
C            definitions. The first call adds the coordinates in the same way
C            that Option 2 does. The second call adds coordinates from
C            a list of user-provided input coordinates in the same way as
C            Option 6x2 does. In other words, there will be two sets of
C            coordinates to be merged. The first set is generated by default
C            (e.g., in the case of Option 9x3 without geom=check) or it comes
C            from a chk file (Option 9x3 with geom=check). The second set is
C            provided by the user as a separate input section, and it contains
C            some additional coordinates or modifications to the first set.
C            A blank line can be used instead of the second set's input
C            section. It is also possible to make the user-provided input's
C            reading to be the first step and the construction of complementary
C            coordinates by CrdCon to be the second step. The latter is
C            Option 6x3.
C      0x ... Which coordinate derivative terms should be included in the
C            data structure? (Default=3x if Hessian is requested, 2x otherwise)
C      1x ... Include only values in the definitions list.
C      2x ... Include only values and their first-derivatives for
C            B-matrix formation.
C      3x ... Include values and first- and second-derivatives for
C            B-matrix and B-matrix derivatives.
C      4x ... Do 3 but turn off smart differentiation (UsSmDq) in CrdDq1
C            (for debugging).
C      0xx ... How should the new code define the coordinate set used when
C            building the data structure? (Default=9xx)
C      1xx ... Use the old coordinate generation code to build IZRed, then
C            have CrdDef build the data structure using IZRed for the
C            coordinates and build derivatives wrt Cartesian
C            coordinates.
C      2xx ... Read the coordinate definitions from the input file using
C            "Format 2" defined in routine CrDf01, and then
C            automatically differentiate the active coordinate
C            derivatives as needed.
C      3xx ... Read the coordinate definitions from the input file using
C            "Format 2" defined in routine CrDf01, and then read in the
C            derivative definitions (one set for IOp(145)=x1x; two sets
C            for IOp(145)=x2x) from the input file using "Format 9"
C            defined in Routine CrDf01.
C      4xx ... Read the coordinate definitions from the input file using
C            "Format 2" defined in routine CrDf01, and then read in the
C            first-derivative definitions using "Format 9" as defined in
C            Routine CrDf01. If second-derivatives are requested
C            (IOp(145)=x2x) then read in the second-derivative
C            definitions using "Format 8" as defined in Routine CrDf01.
C      5xx ... Read the coordinate definitions from the input file using
C            "Format 2" defined in routine CrDf01, and then read in the
C            first-derivative definitions using "Format 8" as defined in
C            Routine CrDf01. If second-derivatives are requested
C            (IOp(145)=x2x) then also read in the second-derivative
C            definitions using "Format 8" as defined in Routine CrDf01.
C      6xx ... Read the coordinate definitions from the input file using
C            the symbolic form (See Routine CrInp1), and then
C            automatically differentiate the active coordinate
C            derivatives as needed.
C      7xx ... Define (the full set of) distance matrix coordinates. This
```

```

C      option can be combined with 2xxx to use inverse distance
C      matrix coordinates.
C      8xx ... Read the coordinate definitions from the input file using
C      the OLD symbolic form (See Routine CrdFSy), and then
C      automatically differentiate the active coordinate
C      derivatives as needed.
C      9xx ... Use the new coordinate generation code to create an intermediate
C      RP structure using CrdCon, then have CrdDef build the data
C      structure using the intermediate RP data and build derivatives
C      wrt Cartesian coordinates. Subroutine CrdCon will use the current
C      connectivity matrix by default, except for a case when
C      IOp(1/156)=5 is requested. If so, the GICs will be obtained by
C      CrdF32 called from CrdDef using the input Z-Matrix.
C      0xxx ... When building the internal coordinate definitions, should
C      any systematic modifications be done? (Default=1)
C      1xxx ... No. Simply define the coordinates to be the same as they
C      would have been using the old coordinate code.
C      2xxx ... Invert all bond stretch coordinates.
C      3xxx ... If the coordinates are GICs generated using the molecular
C      connectivity (See Routine CrdCon), use the input connectivity
C      as is and do not add any new (virtual) bonds by default.
C      1xxxxx ... Optimize the number of GICs for cases when the procedure is
C      implemented.
C      0000000 ... Default for default is 1 (GIC).
C      1000000 ... Default to 1 if not specified.
C      2000000 ... Default to 2 if not specified.

C      IOp(155) ... Control of computing a new set of Cartesian coordinates from
C      the estimated or user-provided values of internal coordinates,
C      e.g. for guessing an initial TS during QST2 or for a new point
C      in a relaxed energy scan calculation (currently works as intended
C      only with GICs):
C      0 ... Default (0000).
C      N ... If 0<N<999, do N passes using the overall step scaled at each
C      pass i by a factor of i/N (i=1, N). If N=0, do one full step
C      (like for N=1) and, if it fails, do N=10.
C      0xxx ... Do a regular least-squares fit of all internal coordinates
C      to their target values (which can be their initial values).
C      1xxx ... Do a partial fitting. Fit the selected internal coordinates
C      to their target values exactly, ignore the remaining ones but
C      keep the Cartesian coordinates at their initial values as much
C      as possible.
C      2xxx ... Do 1xxx but preserve the fragments when moving the atoms provided
C      there are two or more fragments as determined from the default
C      connectivity matrix.

C      IOp(156) ... Automatic generation of internal coordinates.
C      0 ... Default (16 if using GICs).
C      1 ... Generate bonds, angles, and dihedrals.
C      2 ... Generate bonds and angles but no dihedrals.
C      3 ... Generate bonds only.
C      4 ... Generate no coordinates automatically.
C      5 ... Generate Z-Matrix type GICs using opt or geom with gic or
C      addgic and a Z-Matrix provided (GICs only).
C      6 ... Generate the Rotor(1) model GICs for a non-rigid (weakly-bound)
C      system of individual fragments (e.g., a water cluster).
C      The fragment info should be provided via the Fragment= specification
C      or via Fragment() in the GIC input section or via the default or
C      user-provided connectivity information. If there are no fragments
C      then default to 1.
C      7 ... Do 2 and then 6.
C      9 ... Default to 1 and do NoRotors.
C      1x ... Modify the current GICs according to ModelXY3.
C      2x ... Modify the current GICs according to ModelAB3.
C      3x ... Modify the current GICs according to ModelXY3 and ModelAB3.
C      9x ... No modification.
C      1xx ... Symmetrize GICs, saving only totally symmetric.
C      2xx ... Symmetrize GICs, saving all.
C      1xxx ... Use special settings (NTrRot=-3, SkpSLE) to treat coordinate
C      systems that are incomplete (e.g., when we do optimization
C      using only the fully symmetric block of symmetrized GICs).

```

```
C      1xxxx ... Translate and rotate the initial geometry to place the molecular
C      center (XCntr,YCntr,ZCntr) at the origin (0,0,0) and to make the
C      off-diagonal elements of the MofI matrix (X2Cntr,Y2Cntr,Z2Cntr)
C      equal to zero.
```

LIST OF GIC-RELATED TEST EXAMPLES IN \$gdvroot/gdv/tests/com/

test1042.com
test1097.com
test1128.com
test1133.com
test1137.com
test1146.com
test1147.com
test1154.com
test1155.com
test1172.com
test1173.com
test1192.com
test1193.com
test1194.com
test1195.com
test1257.com
test1276.com
test1277.com
test1278.com
test1303.com
test1314.com
test1315.com
test1316.com
test1317.com
test1318.com
test1319.com
test1320.com
test1322.com
test1348.com
test1349.com
test1350.com
test1355.com
test1356.com
test1357.com
test1358.com
test1359.com
test1382.com
test1383.com
test1384.com
test1385.com

test1386.com
test1387.com
test1388.com
test1389.com
test1401.com
test1408.com
test1415.com
test1416.com
test1417.com
test1418.com
test1419.com
test1423.com
test_XXX50_LinAngC60C20.com
test_XXX51_PBCMisc1.com
test_XXX52_PBCMisc2.com
test_XXX53_CrmbX.com
test_XXX54_OniRot.com
test_XXX55_HTransf.com
test_XXX56_pm7b.com
test_XXX57_SN2.com
test_XXX58_consst.com